```
    before(:example) do
      @ts = TennisScorer.new
    end

    it "starts with a score of 0-0" do
      expect(@ts.score).to eq("0-0")
    end

    it "makes the score 15-0 if the server wins a point" do
      @ts.give_point_to(:server)
      expect(@ts.score).to eq("15-0")
    end

    it "raises an error if it doesn't know the player" do
      expect { @ts.give_point_to(:referee) }.to raise_error(RuntimeError)
    end

    it "makes the score 0-15 if the receiver wins a point" do
      @ts.give_point_to(:receiver)
      expect(@ts.score).to eq("0-15")
    end

    it "makes the score 15-15 after they both win a point" do
      @ts.give_point_to(:receiver)
      @ts.give_point_to(:server)
      expect(@ts.score).to eq("15-15")
    end
  end
end
```

Let's run it:

```
$ rspec ts_spec.rb
.....
Finished in 0.00248 seconds (files took 0.05039 seconds to load)
5 examples, 0 failures
```

RSpec gives us an alternative, preferred, way of setting up variables that are conditions for our tests. The let method creates what looks like a variable whose value is given by evaluating a block. This lets us write the following:

unittesting/bdd/5/ts_spec.rb

```
require_relative "tennis_scorer"

RSpec.describe TennisScorer do
  describe "basic scoring" do
    let(:ts) { TennisScorer.new }

    it "starts with a score of 0-0" do
      expect(ts.score).to eq("0-0")
    end

    it "makes the score 15-0 if the server wins a point" do
      ts.give_point_to(:server)
      expect(ts.score).to eq("15-0")
    end

    it "raises an error if it doesn't know the player" do
      expect { ts.give_point_to(:referee) }.to raise_error(RuntimeError)
    end
```